Once the from function is closed, nothing has happened. If you pipe into IO.inspect here, you will see an Ecto.Query struct but no data. We tell Ecto to execute this query using the Repo.all/1 function.

Let's add one more query that's a bit more complex. Type this out at the end of the module:

phone_app/lib/phone_app/conversations/query/sms_message_store.ex

```
def load_message_list do
  distinct_query =
    from(
      m in SmsMessage,
      select: m.id,
      distinct: [m.contact_id],
      order_by: [desc: m.inserted_at]
    )

  from(
    m in SmsMessage,
    where: m.id in subquery(distinct_query),
    order_by: [desc: m.inserted_at],
    preload: [:contact]
  )
  |> Repo.all()
end
```

At a glance, this query looks a lot more complex, but upon closer inspection, it's similar to what you just wrote. There are a few additional keywords that we'll break down.

The line select: m.id results in the SELECT statement generated by Ecto to contain only this one field. This is necessary in our subquery because we only care about comparing the id field. Ecto has a lot of flexibility with how you can use select:. You can use it to append virtual fields to your query, select data into different data structures, and more.

The usage of distinct: [m.contact_id] is exactly what it looks like. This adds a DISTINCT ON clause to the query so that the contact_id field is unique.

This subquery efficiently loads the sidebar view so that only the most recent message for each contact appears. It gets pulled in with where: m.id in subquery(distinct_query). (Of course, you can do this in ActiveRecord as well.)

One benefit of Ecto's query builder DSL is its readability. There's very little magic here. We write queries that are obvious in their functionality at a glance, but we also benefit from Ecto's security and helper functions. There's a bit